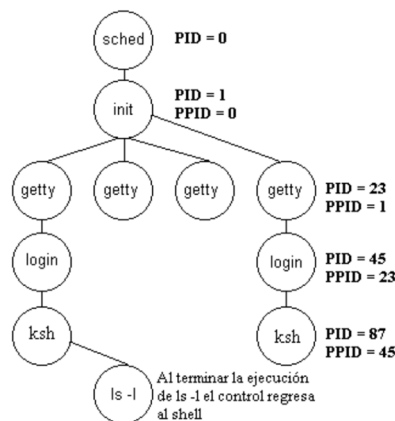


Procesos

Proceso es un concepto fundamental para todo sistema operativo. Es una entidad dinámica que consiste en un programa en ejecución, sus valores actuales, su estado y los recursos utilizados para manejar su ejecución estos son: memoria, CPU, dispositivos de E/S, etc. Pueden coexistir varias instancias de un mismo programa en ejecución en forma simultánea. ya que cada una de ellas es un proceso diferente.

Programa: Un programa es una entidad inactiva, estática, la cual consiste de un conjunto de instrucciones y datos asociados. Si un programa es invocado varias veces puede generar múltiples procesos.

Unix es un sistema multiproceso por tiempo compartido. Aunque en cualquier momento muchos procesos parecen estar ejecutándose concurrentemente desde el punto de vista del proceso, este tiene el acceso y control de todos los recursos del sistema como si fuera único. La realidad es que la mayoría de los sistemas Unix corren en plataformas que tienen una unidad de procesamiento capaz de soportar muchos procesos activos; sin embargo, en un instante dado de tiempo solamente un proceso puede estar en ejecución. Los sistemas operativos Unix cambian rápidamente el proceso actual en ejecución, aparentando así que trabajan en forma concurrente; es decir, aparentan que trabajan en varios procesos al mismo tiempo, pero en realidad existe una distribución temporal de la asignación de CPU entre los diferentes procesos que compiten por ejecutarse. La conmutación temporal de procesos, está basada en un sistema de colas FIFO multinivel con actualización de prioridad. Los sistemas con múltiples unidades de procesamiento, las cuales por definición pueden soportar un verdadero procesamiento concurrente se dice que tienen capacidad de multiprocesamiento. En la figura puede ver que los procesos tienen una estructura jerárquica en árbol:



Tipos de procesos

- **Procesos del sistema:** Son los procesos que actúan sin que el usuario los solicite, también se les conoce como daemons. Pueden ser de dos tipos:
 1. Procesos permanentes o de larga duración: Se crean al arrancar el sistema y permanecen activos hasta que se termina la conexión. Su función es realizar las actividades del sistema.
 2. Procesos transitorios: Nacen y mueren cuando el sistema efectúa tareas propias, independientes de los usuarios.
- **Procesos de usuario:** Son los procesos creados cuando el usuario ejecuta comandos.

Atributos del proceso

- Cada proceso tiene un identificador único (PID). Este es un entero no negativo asignado por el sistema. Garantiza que el proceso sea único dentro del sistema.
- Un proceso tiene asignados tres segmentos de memoria: Segmento de código, Segmento de datos (datos estáticos) y Segmento de Stack (datos dinámicos).
- Identificador del usuario y grupo al que pertenece
- Identificadores de otros procesos relacionados con él
- Datos de los sucesos que le harán despertarse y señales pendientes.
- Tamaño del proceso
- Datos de su planificación.
- Terminal Original.

Prioridades de los Procesos

Una manera de minimizar los tiempos de respuesta del procesador para aprovechar de forma eficiente los recursos del sistema es que mientras un usuario realiza sus procesos otros usuarios pueden tener tareas en ejecución de manera simultánea.

- Dejar trabajos en baja prioridad en el sistema es una buena manera de utilizar los recursos eficientemente.
- Los procesos interactivos (los que requieren entrada/salida) necesitan mayor prioridad.
- El sistema ajusta las prioridades de los procesos dinámicamente, de tal forma que se puede ajustar a cambios en los procesos. Los procesos que no utilizan el procesador por un periodo aumentan su prioridad.

Nice

El comando nice permite ejecutar un proceso en baja prioridad. Dependiendo de la versión de UNIX acepta un rango diferente de prioridades, pero los números más altos tienen prioridad más baja.

nice < -incremento /decremento> <comando>

Ejemplo: \$ nice -1vi

Si no se usa nice la prioridad será establecida por el sistema. Solamente root puede incrementar la prioridad de un proceso.

Estados del proceso

Durante la ejecución de un proceso su estado va cambiando, pudiendo encontrar en cualquiera de los siguientes estados:

- Estado Activo en Ejecución (Running): El proceso tiene asignado un tiempo de CPU y las instrucciones se están ejecutando.
- Estado Activo Ejecutable (Runnable): El proceso puede ser ejecutado, pero no tiene tiempo de CPU disponible.
- Estado Suspendido: Recibió una señal para detenerse (SIGSTOP). Continuará cuando reciba una señal para continuar.

- Durmiendo: Se encuentra en espera de un evento, por ejemplo, una entrada de teclado, o que otros procesos terminen.
- Ocioso(Idle): Fue creado pero aún no es ejecutable.
- Zombi: El proceso terminó pero su padre no ha sido notificado.

Comando ps

El comando ps reporta el estado de los procesos activos.

\$ ps

Si no se especifican opciones, ps mostrará información sobre los procesos que tienen los mismos user ID y controlador de terminal que la conexión desde donde se llama a ps. La información que muestra ps dependerá de las opciones especificadas, por omisión contiene el número de proceso, identificador de terminal, tiempo de ejecución acumulado y el nombre del comando que se está ejecutando.

Modalidades de ps

- a Muestra los procesos de otros usuarios.
- c Muestra el nombre del comando.
- e Lista información sobre cada proceso en ejecución ahora.
- f Muestra más información sobre los procesos. Incluyendo el nombre del usuario.
- l Genera una lista con información detallada de los procesos.
- u user visualiza los procesos de dicho usuario.
- p lista Lista solamente los procesos cuyos números de identificación están en la lista.
- t term Lista los procesos asociados con la terminal. Por ejemplo, term/a, o pts/0.
- u lista Lista los procesos del usuario especificado(s). La lista debe estar separada por comas.
- o formato Muestra la información de acuerdo a un formato especificado en una lista separada por comas.

El formato se especifica usando una o más de las siguientes palabras:

user	ruid	pgid	pcpu	nice	f	psr	comm
ruser	gid	sid	pmem	class	s	tty	args
group	rgid	taskid	vsz	time	c	addr	projid
rgroup	pid	pri	rss	etime	lwp	wchan	project
uid	ppid	opri	osz	stime	nlwp	fname	pset.

Ejemplos: **\$ ps -u root**

\$ps -u root -o pid,pri,nice,time

Cada columna de la información que se visualiza con ps describe el estado del proceso.

- S
O Corriendo
S En espera de un evento sea completado.
I Ocioso.
Z Zombie
T Parado por una señal del padre.

- UID El número ID del usuario.
- PID Número del proceso.
- PPID Número del proceso padre.
- PRI Prioridad del proceso.
- ADDR Dirección de memoria del proceso.
- TIME Tiempo acumulativo de ejecución.
- CMD Nombre del comando. (Nombre completo con -f)
- NI El valor de prioridad proceso.
- SZ El tamaño virtual de la imagen, calculado como el tamaño de texto+pila+datos
- RSS El número de Kilobytes del programa que está residente en la memoria actualmente
- WCHAN El número del evento del kernel por el que está esperando el proceso
- TTY El nombre del terminal de control para el proceso

Ejecución en background y foreground

Desde la línea de comando los procesos pueden crearse de dos maneras:

Primer Plano (foreground): El shell espera a que el proceso termine de ejecutarse para volver a mostrar el prompt.

Ejemplo: **\$cat>tarea**

Segundo Plano (background): El proceso se ejecuta con una prioridad menor, el shell no espera a que termine y mientras tanto el usuario puede ejecutar otro comando. Para mandar una tarea (job) al background el comando se escribe finalizando con un "&". El shell notifica el PID asignado al proceso.

Ejemplo:

```
root@limesurvey ~# vi programa.c&  
[2] 64877
```

Donde [2] es el número de tarea y 64877 el número PID del proceso.

Jobs

A un programa corriendo en el segundo plano o background se le conoce como tarea. El comando Jobs muestra una lista de todos los procesos que un usuario está ejecutando como tarea.

```
root@limesurvey ~# jobs  
[1]-  stopped                  vi programa.c  
[2]+  stopped                  vi programa.c
```

La información anterior indica que la tarea1 (vi) fue detenida

Fg

El comando fg pasa una tarea del background al primer plano. Se especifica el número de tarea que se desea traer al foreground.

```
root@limesurvey ~# fg 1
```

Eliminación de procesos

El comando kill se usa para enviar señales a los procesos. Algunas de estas señales son las siguientes:

kill -HUP <pid> : Señala al proceso con número <pid>, que vuelva a leer sus archivos de configuración

kill -INT <pid> : Señala al proceso con número <pid>, que va a interrumpir.

kill -TERM <pid> : Señala al proceso con numero <pid>,que debe de terminar, a diferencia de -KILL, esta opción da la oportunidad al proceso de terminar.

kill -STOP <pid> : Señala al proceso con numero <pid>, que pare momentaneamente

kill -CONT <pid> : Señala al proceso con numero <pid>, que continúe, este comando se utiliza para reanudar un proceso que le fue aplicado -STOP

kill -KILL <pid> : Señala al proceso con número <pid/notarea>, que termine de inmediato.

Las señales tienen un número asignado en la tabla de señales, en el caso de KILL es 9.

La sintaxis de kill es: **kill <señal> <pid/no.tarea>**

Ejemplo para matar por PID:

```
root@limesurvey ~#  
root@limesurvey ~# kill -9 64961
```

Para eliminar un proceso por número de tarea

```
root@limesurvey ~# kill -9 %2  
[2]+  Stopped                  vi procesos.c
```

El resultado de los ejemplos anteriores es el mismo si se escribe -KILL en vez de 9

```
root@limesurvey ~# kill -KILL 64968  
[3]+  Stopped                  vi tarea.c
```

Comando nohup

Al terminar una sesión de Unix, el sistema mata todos los procesos del usuario. Si algún proceso requiere mayor tiempo de ejecución el comando nohup permite que un proceso continúe ejecutándose aunque el usuario haya terminado su sesión.

Sintaxis:

nohup comando &

```
root@limesurvey ~# nohup find /archivo&  
[1] 64971  
root@limesurvey ~# nohup: ignoring input and appending output to 'nohup.out'
```

Signals

Las señales en Unix son la forma para avisar a un proceso que ha sucedido cierto evento y que debe ser atendido. La lista de señales a comunicar a los procesos varía muy poco entre las versiones de Unix.

Nombre	Valor	Acción	Evento
SIGHUP	1	Exit	Hangup
SIGINT	2	Exit	Interrupt
SIGQUIT	3	Core	Quit
SIGILL	4	Core	Illegal Instruction
SIGTRAP	5	Core	Trace/Breakpoint Trap
SIGABRT	6	Core	Abort
SIGEMT	7	Core	Emulation Trap
SIGFPE	8	Core	Arithmetic Exception
SIGKILL	9	Exit	Killed
SIGBUS	10	Core	Bus Error
SIGSEGV	11	Core	Segmentation Fault
SIGSYS	12	Core	Bad System Call
SIGPIPE	13	Exit	Broken Pipe
SIGALRM	14	Exit	Alarm Clock
SIGTERM	15	Exit	Terminated
SIGUSR1	16	Exit	User Signal 1
SIGUSR2	17	Exit	User Signal 2
SIGCHLD	18	Ignore	Child Status Changed
SIGPWR	19	Ignore	Power Fail/Restart
SIGWINCH	20	Ignore	Window Size Change
SIGURG	21	Ignore	Urgent Socket Condition
SIGPOLL	22	Exit	Pollable Event
SIGIO	22	Exit	input/output possible signal
SIGSTOP	23	Stop	Stopped(signal)
SIGTSTP	24	Stop	Stopped(user)
SIGCONT	25	Ignore	Continued
SIGTTIN	26	Stop	Stopped(tty input)
SIGTTOU	27	Stop	Stopped(tty output)
SIGVTALRM	28	Exit	Virtual Timer Expired
SIGPROF	29	Exit	Profiling Timer Expired
SIGXCPU	30	Core	CPU time limit exceeded
SIGXFSZ	31	Core	File size limit exceeded
SIGCKPT	33	Ignore	Checkpoint warning
SIGRESTART	34	Ignore	Restart warning
SIGRTMIN	49	Exit	POSIX1003.1b SIGRTMIN
SIGRTMAX	64	Exit	POSIX1003.1b SIGRTMAX

Tabla de Señales del sistema operativo UNIX IRIX

1. Genere un listado completo de todos los procesos que están en el sistema y muestre la información completa de todos los que se empezaron a ejecutar el 12 de marzo (en una sola línea).
2. ¿Qué comando son los procesos que actualmente está ejecutando root?
3. Genere un listado con el número de proceso, número del proceso padre, comando en ejecución y prioridad de tres de sus compañeros.
4. Explique la diferencia entre las opciones de ps e, f, l y j
5. Explique la diferencia entre las opciones de ps a y u
6. Explique qué es lo que hace la opción de ps t y u
7. Si tiene dos sesiones de putty abiertas con el mismo user name
 - a. ¿qué procesos muestra al ejecutar ps?
 - b. ¿Qué opción de ps debería de usar para ver todos los procesos de un usuario?
 - c. ¿Cómo identifico a los procesos que el usuario está ejecutando en cada terminal?
8. ¿Cuál es significado de TODAS las columnas de formato que maneja ps -o
9. Ejecute dos comandos cualquiera en background.
10. Ejecute el comando cat >lista, ¿Qué prioridad tiene asignada?
11. Mate el proceso anterior.
12. Vuelva a ejecutar cat>lista pero con menor prioridad.
13. ¿Qué prioridad le fue asignada?
14. Una vez más ejecute cat>lista, pero ahora en el background .
15. ¿Cuál es su prioridad ahora?
16. Verifique que el comando en background este en la lista de procesos.
17. Verifique que el comando en background este en la lista de tareas (jobs).
18. Pase una de las tareas al foreground use el número de tarea.
19. Pase la otra tarea al foreground, pero ahora use el número de PID.
20. Envíe otro comando al background.
21. Finalice este proceso.
22. Compruebe el funcionamiento del comando nohup.